

36. 目的関数重み CEM 学習

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	scripts/tune_upper_planner_weights.py
種別	Python
区分	planning research
役割	複数シナリオで upper planner の cost weight を探索し、risk-aware な candidate を評価する。
起動文脈	learn action の中心。

このファイルを読む理由

複数シナリオで upper planner の cost weight を探索し、risk-aware な candidate を評価する。

- multi-scenario、CVaR、chance gate を扱う。
- operational release ではなく exact validation 前段の性格が強い。

呼び出し関係

どこから呼ばれるか

- scripts/solar_control.sh

次に読むと理解が進むファイル

- mpc_solarcar/upper_cost.py
- scripts/solar_sim.py

同一リポジトリ内の主要依存

- mpc_solarcar/risk_utils.py
- mpc_solarcar/schedule_utils.py
- mpc_solarcar/upper_cost.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
@dataclass
class TermSpec:
    name: str
    lo: float
    hi: float
    init_log10: float
    threshold: float = 1.0e-4

@dataclass
class ScenarioSpec:
    name: str
    cfg_overrides: Dict[str, object]
    cli_overrides: Dict[str, object]
    weight: float

def read_yaml(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as f:
        return yaml.safe_load(f) or {}
```

主要構造の読み取り

主要クラスは TermSpec, ScenarioSpec。主要関数は read_yaml, write_yaml, ensure_dir, log_trial_to_tensorboard, repo_relative, failed_scenario_result, compile_tex, latex_escape。CLI 引数宣言は 19 件。

主要クラス・関数

行	種別	名前	補足
83	class	TermSpec	
92	class	ScenarioSpec	
99	function	read_yaml	args: path
104	function	write_yaml	args: path, payload
110	function	ensure_dir	args: path
114	function	log_trial_to_tensorboard	args: writer, prefix, result, step
147	function	repo_relative	args: path_like
159	function	failed_scenario_result	args: scenario_name, scenario_weight, upper_cost
217	function	compile_tex	args: tex_path
234	function	latex_escape	args: text
253	function	format_override_value	args: value
265	function	canonical_runtime_weights	args: weights

270	function	set_nested_value	args: cfg, dotted_key, value
283	function	speed_series	args: df
291	function	upper_cost_specs	args: cfg
336	function	vector_to_weights	args: specs, vec, base_cfg
347	function	mirror_legacy_weights	args: cfg, upper_cost
360	function	build_reference_free_profile	args: profile_yaml, output_dir
397	function	default_scenarios	args: profile_yaml
482	function	run_single_scenario	args: profile_yaml, output_dir, candidate_name, scenario, upper_cost, cfg_overrides, cli_overrides
613	function	evaluate_simulation	args: profile_yaml, summary, sim_df, detail_df, upper_cost
759	function	aggregate_candidate	args: candidate, scenario_results, weights, risk_config
820	function	run_candidate	args: profile_yaml, output_dir, candidate_name, upper_cost, cfg_overrides, cli_overrides, scenarios, risk_config
858	function	save_trial_checkpoint	args: output_dir, trials, best_result
868	function	csv_row_count	args: path
873	function	summarize_path	args: path
896	function	dataframe_to_markdown	args: df
910	function	flatten_scalars	args: prefix, payload, rows
920	function	build_current_asset_manifests	args: profile_yaml, fit_summary, output_dir
986	function	render_report	args: output_dir, source_profile_yaml, eval_profile_yaml, fit_summary, baseline_result, tuned_result, trials_df, best_weights, manifest_paths, removed_reference, specs, search_cfg, tensorboard_dir
1138	function	tex_path_rel	args: path
1504	function	main	

ファイルを上から読んだときの定義順

行	内容
19	matplotlib.use(...) を実行する。
25	例外処理を伴う try ブロックを実行する。
30	ROOT に Path(__file__).resolve().parents[1] の結果を代入する。
31	条件 str(ROOT) not in sys.path を判定し、真なら内部処理を行う。
39	DEFAULT_PROFILE に ROOT / 'project_packages' / 'bwsc2025_fitted_mle4' / 'profile.yaml' の結果を代入する。

42 LITERATURE に [{ 'label': 'de Boer et al. (2005)', 'title': 'A Tutorial on the Cross-Entropy Method', 'url': 'https://doi.org/10.1007/s10479-005-5724-z', 'note': 'CEM is a generic derivative-free method for hard optimization problems.' }, { 'label': 'Gros and Zanon (2019/2020)', 'title': 'Data-driven Economic NMPC using Reinforcement Learning', 'url': 'https://arxiv.org/pdf/1904.04152', 'note': 'RL can tune stage cost, terminal cost, and constraints of MPC/Economic MPC.' }, { 'label': 'Zarrouki et al. (2024)', 'title': 'A Safe Reinforcement Learning driven Weights-varying Model Predictive Controller', 'url': 'https://arxiv.org/pdf/2402.02624', 'note': 'Safe RL can adapt MPC weights within a restricted safe search space.' }, { 'label': 'Howlett et al. (1997)', 'title': 'Optimal driving strategy for a solar car on a level road', 'url': 'https://doi.org/10.1093/imaman/8.1.59', 'note': 'Solar-race strategy is governed by a tight energy-speed trade-off.' }, { 'label': 'Pudney and Howlett (2002)', 'title': 'Critical Speed Control of a Solar Car', 'url': 'https://link.springer.com/article/10.1023/A%3A1020907101234', 'note': 'Large unnecessary speed deviations are undesirable in solar-race operation.' }, { 'label': 'Byrd et al. (1995)', 'title': 'A Limited Memory Algorithm for Bound Constrained Optimization', 'url': 'https://doi.org/10.1137/0916069', 'note': 'L-BFGS-B provides bounded local refinement after global candidate search.' }] の結果を代入する。

83 クラス TermSpec を定義する。

92 クラス ScenarioSpec を定義する。

99 関数 read_yaml を定義する。

104 関数 write_yaml を定義する。

110 関数 ensure_dir を定義する。

114 関数 log_trial_to_tensorboard を定義する。

147 関数 repo_relative を定義する。

159 関数 failed_scenario_result を定義する。

217 関数 compile_tex を定義する。

234 関数 latex_escape を定義する。

253 関数 format_override_value を定義する。

265 関数 canonical_runtime_weights を定義する。

270 関数 set_nested_value を定義する。

283 関数 speed_series を定義する。

291 関数 upper_cost_specs を定義する。

336 関数 vector_to_weights を定義する。

347 関数 mirror_legacy_weights を定義する。

360 関数 build_reference_free_profile を定義する。

397 関数 default_scenarios を定義する。

482 関数 run_single_scenario を定義する。

613 関数 evaluate_simulation を定義する。

759 関数 aggregate_candidate を定義する。

820 関数 run_candidate を定義する。

858 関数 save_trial_checkpoint を定義する。

868 関数 csv_row_count を定義する。

873 関数 summarize_path を定義する。

896 関数 dataframe_to_markdown を定義する。

910 関数 flatten_scalars を定義する。

920 関数 build_current_asset_manifests を定義する。

986 関数 render_report を定義する。

1504 関数 main を定義する。

1940 条件 __name__ == '__main__' を判定し、真なら内部処理を行う。

import 群の詳細

行	import 文	このファイルで読む理由
2	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
4	<code>importargparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L1505。
5	<code>importjson</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L571, L607, L852, L1936, L1937。
6	<code>importmath</code>	clip、sqrt、sin/cos、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L259, L299, L300, L301, L302, L303, L304, L305, ...。
7	<code>importos</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L154, L205, L206, L207, L208, L209, L210, L211, ...。
8	<code>importshutil</code>	shutil モジュールを利用するため。このファイル内での主な使用位置は L1862。
9	<code>importsubprocess</code>	git 情報取得や外部コマンド実行を行うため。このファイル内での主な使用位置は L220, L224, L225, L228, L545, L550。
10	<code>importsys</code>	sys モジュールを利用するため。このファイル内での主な使用位置は L31, L32, L504。
11	<code>importtextwrap</code>	textwrap モジュールを利用するため。このファイル内での主な使用位置は L1444。
12	<code>fromdataclassesimportdataclass</code>	dataclasses から dataclass を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L82, L91。
13	<code>fromdatetimeimportdatetime</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L965, L1163, L1563。
14	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L30, L99, L104, L110, L151, L167, L168, L169, ...。
15	<code>fromtypingimportDict, Iterable,List</code>	typing から Dict, Iterable, List を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L94, L95, L114, L162, L165, L166, L174, L265, ...。
17	<code>importmatplotlib</code>	matplotlib モジュールを利用するため。このファイル内での主な使用位置は L19。
20	<code>importmatplotlib. pyplotasplt</code>	matplotlib.pyplot モジュールを利用するため。このファイル内での主な使用位置は L1037, L1038, L1039, L1040, L1041, L1042, L1043, L1044, ...。
21	<code>importnumpyasnp</code>	数値配列、clip、補間、統計計算を行うため。このファイル内での主な使用位置は L288, L336, L339, L627, L637, L639, L643, L653, ...。
22	<code>importpandasaspd</code>	CSV や時刻列の読み込み・整形・再サンプリングに使うため。このファイル内での主な使用位置は L283, L288, L572, L573, L616, L617, L623, L626, ...。

23	<code>importyaml</code>	<code>profile</code> 、 <code>stop</code> 、 <code>schedule</code> 、 <code>summary</code> YAML を読み書きするため。このファイル内での主な使用位置は L101, L107。
26	<code>fromtorch.utils.tensorboardimportSummaryWriter</code>	<code>torch.utils.tensorboard</code> から <code>SummaryWriter</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L28, L1567。
34	<code>frommpc_solarcar.schedule_utilsimportDriveSchedule</code>	<code>schedule_utils.py</code> から <code>DriveSchedule</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/schedule_utils.py</code> 。このファイル内での主な使用位置は L635。
35	<code>frommpc_solarcar.risk_utilsimportScenarioRiskConfig, aggregate_scenario_scores</code>	<code>risk_utils.py</code> から <code>ScenarioRiskConfig</code> , <code>aggregate_scenario_scores</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/risk_utils.py</code> 。このファイル内での主な使用位置は L763, L775, L828, L1536。
36	<code>frommpc_solarcar.upper_costimportUpperCostConfig, active_upper_cost_terms, load_upper_cost_config</code>	上位 MPC 目的関数から <code>UpperCostConfig</code> , <code>active_upper_cost_terms</code> , <code>load_upper_cost_config</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/upper_cost.py</code> 。このファイル内での主な使用位置は L175, L292, L336, L684, L1135, L1534, L1552。

関数・クラスを上から順に解説

L83 クラス `TermSpec`

- 定義: `TermSpec(bases=□□)`
 - `docstring`: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
 - 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
1. `name` にを代入する。
 2. `lo` にを代入する。
 3. `hi` にを代入する。
 4. `init_log10` にを代入する。
 5. `threshold` に `0.0001` を代入する。

L92 クラス `ScenarioSpec`

- 定義: `ScenarioSpec(bases=□□)`
 - `docstring`: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
 - 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
1. `name` にを代入する。
 2. `cfg_overrides` にを代入する。

3. `cli_overrides` にを代入する。
4. `weight` にを代入する。

L99 関数 `read_yaml`

- 定義: `read_yaml(path)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `open`, `safe_load`
 - 戻り値の要点: `yaml.safe_load(f)` or `{}`
1. `with` 文で `path.open('r', encoding='utf-8')` を管理しながら処理する。

L104 関数 `write_yaml`

- 定義: `write_yaml(path, payload)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `mkdir`, `open`, `safe_dump`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `path.parent.mkdir(...)` を実行する。
 2. `with` 文で `path.open('w', encoding='utf-8', newline='\n')` を管理しながら処理する。

L110 関数 `ensure_dir`

- 定義: `ensure_dir(path)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `mkdir`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `path.mkdir(...)` を実行する。

L114 関数 `log_trial_to_tensorboard`

- 定義: `log_trial_to_tensorboard(writer, prefix, result, step)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `add_scalar`, `float`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. 条件 `writer is None` を判定し、真なら内部処理を行う。
 2. `scalar_keys` に `['score', 'score_mean', 'score_worst', 'score_variance', 'loss_cvar', 'scenario_failure_probability', 'chance_constraint_pass', 'final_distance_km', 'final_distance_worst_km', 'avg_speed_kmh', 'min_soc', 'final_soc', 'oscillation_mean_abs_dv_kmh', 'oscillation_p95_abs_dv_kmh', 'current_rms_a', 'pack_slew_rms_kw', 'daylight_stop_h', 'daylight_full_soc_h', 'unused_finish_soc', 'cpu_sec', 'active_term_count']` の結果を代入する。
 3. `scalar_keys` を順に走査し、各要素を `key` に入れて処理する。

L147 関数 `repo_relative`

- 定義: `repo_relative(path_like)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Path`, `fspath`, `is_absolute`, `relative_to`, `replace`, `resolve`, `str`, `strip`
 - 戻り値の要点: `"/os.fspath(resolved.relative_to(ROOT)).replace('\', '/')/raw.replace('\', '/')`
1. `raw` に `str(path_like or "/").strip()` の結果を代入する。
 2. 条件 `not raw` を判定し、真なら内部処理を行う。
 3. `path` に `Path(raw)` の結果を代入する。
 4. 例外処理を伴う `try` ブロックを実行する。

L159 関数 `failed_scenario_result`

- 定義: `failed_scenario_result(scenario_name, scenario_weight, upper_cost, error, cfg_overrides, cli_overrides, summary_json, out_csv, detail_csv, plan_csv, report_html, resolved_yaml, sim_log_path)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `UpperCostConfig`, `active_upper_cost_terms`, `float`, `fspath`, `int`, `len`
 - 戻り値の要点: `{'score': -1000000000.0, 'final_distance_km': 0.0, 'avg_speed_kmh': 0.0, 'min_soc': 0.0, 'final_soc': 0.0, 'elapsed_hours': 0.0, 'cpu_sec': 0.0, 'finish_reached': False, 'model_validation_gate_pass': False, 'scenario_feasible': False, 'scenario_feasibility_checks': {'simulation_completed': False}, 'scenario_infeasibility_reasons': [error], 'oscillation_mean_abs_dv_kmh': 1000000.0, 'oscillation_p95_abs_dv_kmh': 1000000.0, 'current_rms_a': 1000000.0, 'pack_slew_rms_kw': 1000000.0, 'high_speed_h': 1000000.0, 'daylight_stop_h': 1000000.0, 'daylight_full_soc_h': 1000000.0, 'unused_finish_soc': 1.0, 'finish_soc_target': 0.0, 'terminal_soc_error': 1.0, 'active_term_count': int(len(active_terms)), 'active_terms': active_terms, 'scenario': scenario_name, 'scenario_weight': float(scenario_weight), 'cfg_overrides': cfg_overrides, 'cli_overrides': cli_overrides, 'summary_json': os.fspath(summary_json), 'out_csv': os.fspath(out_csv), 'detail_csv': os.fspath(detail_csv), 'plan_csv': os.fspath(plan_csv), 'report_html': os.fspath(report_html), 'resolved_yaml': os.fspath(resolved_yaml), 'simulation_log': os.fspath(sim_log_path), 'failed': True, 'error': error}`
1. `active_terms` に `active_upper_cost_terms(UpperCostConfig(**upper_cost), threshold=0.0001)` の結果を代入する。
 2. `{'score': -1000000000.0, 'final_distance_km': 0.0, 'avg_speed_kmh': 0.0, 'min_soc': 0.0, 'final_soc': 0.0, 'elapsed_hours': 0.0, 'cpu_sec': 0.0, 'finish_reached': False, 'model_validation_gate_pass': False, 'scenario_feasible': False, 'scenario_feasibility_checks': {'simulation_completed': False}, 'scenario_infeasibility_reasons': [error], 'oscillation_mean_abs_dv_kmh': 1000000.0, 'oscillation_p95_abs_dv_kmh': 1000000.0, 'current_rms_a': 1000000.0, 'pack_slew_rms_kw': 1000000.0, 'high_speed_h': 1000000.0, 'daylight_stop_h': 1000000.0, 'daylight_full_soc_h': 1000000.0, 'unused_finish_soc': 1.0, 'finish_soc_target': 0.0, 'terminal_soc_error': 1.0, 'active_term_count': int(len(active_terms)), 'active_terms': active_terms, 'scenario': scenario_name, 'scenario_weight': float(scenario_weight), 'cfg_overrides': cfg_overrides, 'cli_overrides': cli_overrides, 'summary_json': os.fspath(summary_json), 'out_csv': os.fspath(out_csv), 'detail_csv': os.fspath(detail_csv), 'plan_csv': os.fspath(plan_csv), 'report_html': os.fspath(report_html), 'resolved_yaml': os.fspath(resolved_yaml), 'simulation_log': os.fspath(sim_log_path), 'failed': True, 'error': error}` を返す。

L217 関数 `compile_tex`

- 定義: `compile_tex(tex_path)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `CalledProcessError`, `FileNotFoundError`, `exists`, `range`, `run`, `with_suffix`
 - 戻り値の要点: `pdf_path`
1. `pdf_path` に `tex_path.with_suffix('.pdf')` の結果を代入する。
 2. `range(2)` を順に走査し、各要素を `_` に入れて処理する。
 3. 条件 `not pdf_path.exists()` を判定し、真なら内部処理を行う。
 4. `pdf_path` を返す。

L234 関数 `latex_escape`

- 定義: `latex_escape(text)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `items`, `replace`, `str`
 - 戻り値の要点: `out`
1. `repl` に `{'\\': '\\textbackslash{ }', '&': '\\&', '%': '\\%', '$': '\\$', '#': '\\#', '_': '\\_', '{': '\\{', '}': '\\}', '~': '\\textasciitilde{ }', '^': '\\textasciicircum{ }'}` の結果を代入する。
 2. `out` に `str(text)` の結果を代入する。
 3. `repl.items()` を順に走査し、各要素を `(src, dst)` に入れて処理する。
 4. `out` を返す。

L253 関数 `format_override_value`

- 定義: `format_override_value(value)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `isfinite`, `isinstance`, `str`
 - 戻り値の要点: `str(value) / 'true' if value else 'false' / str(value) / '0'`
1. 条件 `isinstance(value, bool)` を判定し、真なら内部処理を行う。
 2. 条件 `isinstance(value, int)` を判定し、真なら内部処理を行う。
 3. 条件 `isinstance(value, float)` を判定し、真なら内部処理を行う。
 4. `str(value)` を返す。

L265 関数 `canonical_runtime_weights`

- 定義: `canonical_runtime_weights(weights)`
- docstring: Return the exact numeric values serialized into solar_sim overrides.
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `format_override_value`, `items`, `str`
- 戻り値の要点: `{str(key): float(format_override_value(float(value))) for key, value in weights.items()}`

1. `{str(key): float(format_override_value(float(value))) for key, value in weights.items()}` を返す。

L270 関数 `set_nested_value`

- 定義: `set_nested_value(cfg, dotted_key, value)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `get`, `isinstance`, `split`, `str`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。

1. `target` に `cfg` の結果を代入する。

2. `parts` に `[part for part in str(dotted_key).split('.') if part]` の結果を代入する。

3. `parts[:-1]` を順に走査し、各要素を `part` に入れて処理する。

4. 条件 `parts` を判定し、真なら内部処理を行う。

L283 関数 `speed_series`

- 定義: `speed_series(df)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Series`, `astype`, `len`, `zeros`
- 戻り値の要点: `pd.Series(np.zeros(len(df), dtype=float)) / df['v_exec_kmh'].astype(float) / df['v_cmd_kmh'].astype(float)`

1. 条件 `'v_exec_kmh' in df.columns` を判定し、真なら内部処理を行う。

2. 条件 `'v_cmd_kmh' in df.columns` を判定し、真なら内部処理を行う。

3. `pd.Series(np.zeros(len(df), dtype=float))` を返す。

L291 関数 `upper_cost_specs`

- 定義: `upper_cost_specs(cfg, include_progress_terms, include_uncertainty_term, include_terminal_term)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `TermSpec`, `append`, `extend`, `log10`, `max`
- 戻り値の要点: `specs`

1. specs に [TermSpec('w_speed_smooth', -2.0, 3.0, math.log10(max(cfg.w_speed_smooth, 1e-06))), TermSpec('w_speed_limit', -2.0, 3.0, math.log10(max(cfg.w_speed_limit, 1e-06))), TermSpec('w_current_sq', -5.0, 1.0, math.log10(max(cfg.w_current_sq, 1e-06))), TermSpec('w_pack_energy', -5.0, 2.0, math.log10(max(cfg.w_pack_energy, 1e-06))), TermSpec('w_joule_loss', -5.0, 2.0, math.log10(max(cfg.w_joule_loss, 1e-06))), TermSpec('w_aero_energy', -5.0, 2.0, math.log10(max(cfg.w_aero_energy, 1e-06))), TermSpec('w_mech_energy', -5.0, 2.0, math.log10(max(cfg.w_mech_energy, 1e-06))), TermSpec('w_kinetic_pos', -5.0, 2.0, math.log10(max(cfg.w_kinetic_pos, 1e-06))), TermSpec('w_pack_power_slew', -4.0, 4.0, math.log10(max(cfg.w_pack_power_slew, 1e-06))), TermSpec('w_speed_quartic', -6.0, 2.0, math.log10(max(cfg.w_speed_quartic, 1e-06))), TermSpec('w_solar_headroom', -2.0, 7.0, max(2.0, math.log10(max(cfg.w_solar_headroom, 1e-06)))), TermSpec('w_soc_floor_barrier', -6.0, 4.0, math.log10(max(cfg.w_soc_floor_barrier, 1e-06))), TermSpec('w_terminal_soc_min', -1.0, 4.0, math.log10(max(cfg.w_terminal_soc_min, 1e-06))), TermSpec('w_day_end_soc_min', 2.0, 6.0, math.log10(max(cfg.w_day_end_soc_min, 1e-06)))] の結果を代入する。
2. 条件 include_uncertainty_term を判定し、真なら内部処理を行う。
3. 条件 include_terminal_term を判定し、真なら内部処理を行う。
4. 条件 include_progress_terms を判定し、真なら内部処理を行う。
5. specs を返す。

L336 関数 vector_to_weights

- 定義: vector_to_weights(specs, vec, base_cfg)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: clip, float, to_dict, zip
- 戻り値の要点: weights

1. weights に base_cfg.to_dict() の結果を代入する。
2. zip(specs, vec) を順に走査し、各要素を (spec, raw) に入れて処理する。
3. weights を返す。

L347 関数 mirror_legacy_weights

- 定義: mirror_legacy_weights(cfg, upper_cost)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float, get, setdefault
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。

1. mpc に cfg.setdefault('mpc', {}) の結果を代入する。
2. mpc['upper_cost'] に upper_cost の結果を代入する。
3. mpc['w_dv'] に float(upper_cost.get('w_speed_smooth', mpc.get('w_dv', 30.0))) の結果を代入する。
4. mpc['w_dv_limit'] に float(upper_cost.get('w_dv_limit', mpc.get('w_dv_limit', 2.0))) の結果を代入する。
5. mpc['w_speed_limit'] に float(upper_cost.get('w_speed_limit', mpc.get('w_speed_limit', 50.0))) の結果を代入する。
6. mpc['w_current'] に float(upper_cost.get('w_current_sq', mpc.get('w_current', 0.01))) の結果を代入する。
7. mpc['w_T'] に float(upper_cost.get('w_temp', mpc.get('w_T', 5.0))) の結果を代入する。

8. `mpc['w_soc_day_max']` に `float(upper_cost.get('w_soc_day_max', mpc.get('w_soc_day_max', 10000.0)))` の結果を代入する。
9. `mpc['w_soc_day_track']` に `float(upper_cost.get('w_soc_day_track', mpc.get('w_soc_day_track', 0.0)))` の結果を代入する。
10. `mpc['w_soc_terminal']` に `float(upper_cost.get('w_soc_terminal', mpc.get('w_soc_terminal', 0.0)))` の結果を代入する。

L360 関数 `build_reference_free_profile`

- 定義: `build_reference_free_profile(profile_yaml, output_dir, disable_uncertainty_reserve)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Path`, `append`, `fspath`, `is_absolute`, `isinstance`, `items`, `list`, `pop`, `read_yaml`, `resolve`, `setdefault`, `str`
 - 戻り値の要点: (`out_path`, `removed_reference`)
1. `cfg` に `read_yaml(profile_yaml)` の結果を代入する。
 2. `removed_reference` に” の結果を代入する。
 3. `paths` に `cfg.setdefault('paths', {})` の結果を代入する。
 4. 条件 `isinstance(paths, dict)` を判定し、真なら内部処理を行う。
 5. `meta` に `cfg.setdefault('meta', {})` の結果を代入する。
 6. `notes` に `meta.setdefault('notes', [])` の結果を代入する。
 7. 条件 `isinstance(notes, list)` を判定し、真なら内部処理を行う。
 8. `mpc` に `cfg.setdefault('mpc', {})` の結果を代入する。
 9. `ref_cfg` に `mpc.setdefault('reference_speed_tracking', {})` の結果を代入する。
 10. 条件 `isinstance(ref_cfg, dict)` を判定し、真なら内部処理を行う。

L397 関数 `default_scenarios`

- 定義: `default_scenarios(profile_yaml, mode)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ScenarioSpec`, `float`, `get`, `isinstance`, `lower`, `max`, `min`, `read_yaml`, `str`, `strip`
- 戻り値の要点: [`ScenarioSpec('nominal', cfg_overrides={}, cli_overrides={}, weight=0.3)`, `ScenarioSpec('low_solar_high', cfg_overrides={'model.P_aux': max(20.0, base_aux + 20.0), 'model.CdA': max(base_cda * 1.1, base_cda + 0.005), 'model.Crr': max(base_crr * 1.05, base_crr + 0.0002)}, cli_overrides={'solar_gain': 0.9, 'poa_gain_drive': 0.94, 'poa_gain_stop': 0.92}, weight=0.2)`, `ScenarioSpec('drag_bias', cfg_overrides={'model.P_aux': max(10.0, base_aux + 10.0), 'model.CdA': max(base_cda * 1.2, base_cda + 0.01), 'model.Crr': max(base_crr * 1.08, base_crr + 0.0004)}, cli_overrides={'solar_gain': 0.97, 'poa_gain_drive': 0.98, 'poa_gain_stop': 0.98}, weight=0.1)`, `ScenarioSpec('low_initial_soc', cfg_overrides={}, cli_overrides={'soc0': max(soc_min + 0.04, base_soc - 0.15)}, weight=0.15)`, `ScenarioSpec('hot_battery', cfg_overrides={}, cli_overrides={'Tb0': min(temp_max_c - 2.0, base_tb_c + 12.0)}, weight=0.1)`, `ScenarioSpec('driver_lag', cfg_overrides={}, cli_overrides={'exec_tau_sec': 4.0, 'exec_reaction_delay': 3.0, 'exec_accel_limit_kmhps': 0.7, 'exec_decel_limit_kmhps': 2.0}, weight=0.1)`, `ScenarioSpec('favorable_weather', cfg_overrides={'model.P_aux': max(0.0, base_aux - 10.0)}, cli_overrides={'solar_gain': 1.08, 'poa_gain_drive': 1.04, 'poa_gain_stop': 1.04}, weight=0.05)] / [ScenarioSpec('nominal', cfg_overrides={}, cli_overrides={}, weight=1.0)]`

1. 条件 `str(mode).strip().lower() == 'nominal'` を判定し、真なら内部処理を行う。
2. `cfg` に `read_yaml(profile_yaml)` の結果を代入する。
3. `model` に `cfg.get('model', {})` if `isinstance(cfg, dict)` else `{}` の結果を代入する。
4. `base_cda` に `float(model.get('CdA', 0.08))` の結果を代入する。
5. `base_crr` に `float(model.get('Crr', 0.008))` の結果を代入する。
6. `base_aux` に `float(model.get('P_aux', 0.0))` の結果を代入する。
7. `simulation` に `cfg.get('simulation', {})` if `isinstance(cfg, dict)` else `{}` の結果を代入する。
8. `base_soc` に `float(simulation.get('soc0', 0.99))` の結果を代入する。
9. `base_tb_c` に `float(simulation.get('Tb0', 30.0))` の結果を代入する。
10. `soc_min` に `float(model.get('soc_min', 0.05))` の結果を代入する。

L482 関数 `run_single_scenario`

- 定義: `run_single_scenario(profile_yaml, output_dir, candidate_name, scenario, upper_cost, cfg_overrides, cli_overrides)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `Path`, `dict`, `dumps`, `ensure_dir`, `evaluate_simulation`, `exists`, `extend`, `failed_scenario_result`, `float`, `format_override_value`, `fspath`
- 戻り値の要点: `result`

1. `scenario_dir` に `output_dir / 'candidates' / candidate_name / scenario.name` の結果を代入する。
2. `ensure_dir(...)` を実行する。
3. `out_csv` に `scenario_dir / 'simulation.csv'` の結果を代入する。
4. `detail_csv` に `scenario_dir / 'simulation_detail.csv'` の結果を代入する。
5. `plan_csv` に `scenario_dir / 'upper_plan.csv'` の結果を代入する。
6. `report_html` に `scenario_dir / 'simulation_report.html'` の結果を代入する。
7. `summary_json` に `scenario_dir / 'summary.json'` の結果を代入する。
8. `resolved_yaml` に `scenario_dir / 'resolved.yaml'` の結果を代入する。
9. `latest_manifest_json` に `scenario_dir / 'latest_manifest.json'` の結果を代入する。
10. `sim_log_path` に `scenario_dir / 'solar_sim_console.log'` の結果を代入する。

L613 関数 `evaluate_simulation`

- 定義: `evaluate_simulation(profile_yaml, summary, sim_df, detail_df, upper_cost)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Series`, `UpperCostConfig`, `abs`, `active_upper_cost_terms`, `astype`, `bool`, `diff`, `exists`, `fillna`, `float`, `from_yaml`, `fspath`

- 戻り値の要点: {'score': float(score), 'final_distance_km': float(summary['final_distance_km']), 'avg_speed_kmh': float(summary.get('avg_speed_kmh', 0.0)), 'min_soc': min_soc, 'final_soc': final_soc, 'elapsed_hours': elapsed_hours, 'cpu_sec': float(summary.get('cpu_sec', 0.0)), 'finish_reached': finish_reached, 'scenario_feasible': not infeasibility_reasons, 'scenario_feasibility_checks': feasibility_checks, 'scenario_infeasibility_reasons': infeasibility_reasons, 'oscillation_mean_abs_dv_kmh': osc, 'oscillation_p95_abs_dv_kmh': osc_p95, 'current_rms_a': current_rms_a, 'pack_slew_rms_kw': pack_slew_rms_kw, 'high_speed_h': high_speed_h, 'daylight_stop_h': daylight_stop_h, 'daylight_full_soc_h': daylight_full_soc_h, 'unused_finish_soc': unused_finish_soc, 'finish_soc_target': finish_soc_target, 'terminal_soc_error': terminal_soc_error, 'active_term_count': int(len(active_terms)), 'active_terms': active_terms}

1. speed_vals に speed_series(sim_df).to_numpy(dtype=float) の結果を代入する。
2. 条件 'time_utc' in sim_df.columns and len(sim_df) >= 2 を判定し、真なら内部処理を行う。
3. profile_cfg に read_yaml(profile_yaml) の結果を代入する。
4. schedule に None の結果を代入する。
5. schedule_rel に ((profile_cfg.get('paths', {})) if isinstance(profile_cfg, dict) else {}) or {}).get('drive_schedule_yaml') の結果を代入する。
6. 条件 schedule_rel を判定し、真なら内部処理を行う。
7. daylight_mask に sim_df.get('G_poa', pd.Series(np.zeros(len(sim_df), dtype=float))).astype(float) >= 250.0 の結果を代入する。
8. stopped_mask に speed_series(sim_df) <= 1.0 の結果を代入する。
9. soc_mask に sim_df.get('soc', pd.Series(np.zeros(len(sim_df), dtype=float))).astype(float) >= 0.95 の結果を代入する。
10. 条件 schedule is not None and len(t_series) == len(sim_df) を判定し、真なら内部処理を行う。

L759 関数 aggregate_candidate

- 定義: aggregate_candidate(candidate, scenario_results, weights, risk_config)
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: ValueError, aggregate_scenario_scores, all, array, asarray, bool, dot, float, get, int, isfinite, len
 - 戻り値の要点: result
1. 条件 not scenario_results を判定し、真なら内部処理を行う。
 2. scenario_weights に np.array([max(0.0, float(row.get('scenario_weight', 0.0))) for row in scenario_results], dtype=float) の結果を代入する。
 3. 条件 not np.isfinite(scenario_weights).all() or scenario_weights.sum() <= 0.0 を判定し、真なら内部処理を行う。
 4. scenario_scores に np.array([float(row['score']) for row in scenario_results], dtype=float) の結果を代入する。
 5. scenario_feasible に np.array([bool(row.get('scenario_feasible', False)) for row in scenario_results], dtype=bool) の結果を代入する。
 6. risk に aggregate_scenario_scores(scenario_scores, scenario_weights, scenario_feasible, config=risk_config) の結果を代入する。
 7. scenario_weights に np.asarray(risk['normalized_scenario_weights'], dtype=float) の結果を代入する。

8. `nominal` に `next((row for row in scenario_results if row.get('scenario') == 'nominal'), scenario_results[0])` の結果を代入する。
9. `result` に `{'candidate': candidate, **risk, 'final_distance_km': float(np.dot(scenario_weights, np.array([float(row['final_distance_km']) for row in scenario_results]))), 'final_distance_worst_km': float(min((float(row['final_distance_km']) for row in scenario_results))), 'avg_speed_kmh': float(np.dot(scenario_weights, np.array([float(row['avg_speed_kmh']) for row in scenario_results]))), 'min_soc': float(min((float(row['min_soc']) for row in scenario_results))), 'final_soc': float(np.dot(scenario_weights, np.array([float(row['final_soc']) for row in scenario_results]))), 'finish_reached': bool(all((bool(row['finish_reached']) for row in scenario_results))), 'model_validation_gate_pass_all': bool(all((bool(row.get('model_validation_gate_pass', False)) for row in scenario_results))), 'oscillation_mean_abs_dv_kmh': float(np.dot(scenario_weights, np.array([float(row['oscillation_mean_abs_dv_kmh']) for row in scenario_results]))), 'oscillation_p95_abs_dv_kmh': float(np.dot(scenario_weights, np.array([float(row['oscillation_p95_abs_dv_kmh']) for row in scenario_results]))), 'current_rms_a': float(np.dot(scenario_weights, np.array([float(row['current_rms_a']) for row in scenario_results]))), 'pack_slew_rms_kw': float(np.dot(scenario_weights, np.array([float(row['pack_slew_rms_kw']) for row in scenario_results]))), 'high_speed_h': float(np.dot(scenario_weights, np.array([float(row['high_speed_h']) for row in scenario_results]))), 'daylight_stop_h': float(np.dot(scenario_weights, np.array([float(row['daylight_stop_h']) for row in scenario_results]))), 'daylight_full_soc_h': float(np.dot(scenario_weights, np.array([float(row['daylight_full_soc_h']) for row in scenario_results]))), 'unused_finish_soc': float(np.dot(scenario_weights, np.array([float(row['unused_finish_soc']) for row in scenario_results]))), 'finish_soc_target': float(np.dot(scenario_weights, np.array([float(row['finish_soc_target']) for row in scenario_results]))), 'terminal_soc_error': float(np.dot(scenario_weights, np.array([float(row['terminal_soc_error']) for row in scenario_results]))), 'elapsed_hours': float(np.dot(scenario_weights, np.array([float(row['elapsed_hours']) for row in scenario_results]))), 'cpu_sec': float(sum((float(row['cpu_sec']) for row in scenario_results))), 'active_term_count': int(round(np.dot(scenario_weights, np.array([float(row['active_term_count']) for row in scenario_results]))), 'weights': weights, 'scenario_results': scenario_results, 'nominal_out_csv': nominal.get('out_csv', ''), 'nominal_detail_csv': nominal.get('detail_csv', ''))} の結果を代入する。`
10. `result` を返す。

L820 関数 `run_candidate`

- 定義: `run_candidate(profile_yaml, output_dir, candidate_name, upper_cost, cfg_overrides, cli_overrides, scenarios, risk_config)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `aggregate_candidate`, `canonical_runtime_weights`, `dumps`, `ensure_dir`, `run_single_scenario`, `write_text`
- 戻り値の要点: `result`

1. `runtime_upper_cost` に `canonical_runtime_weights(upper_cost)` の結果を代入する。
2. `scenario_results` に `[run_single_scenario(profile_yaml, output_dir, candidate_name, scenario, runtime_upper_cost, cfg_overrides, cli_overrides) for scenario in scenarios]` の結果を代入する。
3. `result` に `aggregate_candidate(candidate_name, scenario_results, runtime_upper_cost, risk_config=risk_config)` の結果を代入する。
4. `cand_dir` に `output_dir / 'candidates' / candidate_name` の結果を代入する。
5. `ensure_dir(...)` を実行する。
6. `(cand_dir / 'aggregate_metrics.json').write_text(...)` を実行する。
7. `result` を返す。

L858 関数 `save_trial_checkpoint`

- 定義: `save_trial_checkpoint(output_dir, trials, best_result)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `float`, `get`, `to_csv`, `write_yaml`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。

1. 条件 `trials` を判定し、真なら内部処理を行う。
2. 条件 `best_result is not None` を判定し、真なら内部処理を行う。

L868 関数 `csv_row_count`

- 定義: `csv_row_count(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `max`, `open`, `sum`
- 戻り値の要点: `max(0, sum((1 for _ in f)) - 1)`

1. `with` 文で `path.open('r', encoding='utf-8', errors='ignore')` を管理しながら処理する。

L873 関数 `summarize_path`

- 定義: `summarize_path(path)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `csv_row_count`, `exists`, `fspath`, `join`, `len`, `lower`, `lstrip`, `read_csv`, `str`
- 戻り値の要点: `summary / summary`

1. `suffix` に `path.suffix.lower()` の結果を代入する。
2. `summary` に `{'path': os.fspath(path), 'exists': path.exists(), 'kind': suffix.lstrip('.'), 'rows': '', 'columns': '', 'column_names': ''}` の結果を代入する。
3. 条件 `not path.exists()` を判定し、真なら内部処理を行う。
4. 条件 `suffix == '.csv'` を判定し、真なら内部処理を行う。
5. `summary` を返す。

L896 関数 `dataframe_to_markdown`

- 定義: `dataframe_to_markdown(df)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `iterrows`, `join`, `len`, `replace`, `str`
- 戻り値の要点: `'\n'.join(lines) / '(none)'`

1. 条件 `df.empty` を判定し、真なら内部処理を行う。

2. cols に [str(col) for col in df.columns] の結果を代入する。
3. lines に ['|'+ '|'.join(cols)+'|', '|'+ '|'.join(['—'] * len(cols))+'|'] の結果を代入する。
4. df.iterrows() を順に走査し、各要素を (_, row) に入れて処理する。
5. '\n'.join(lines) を返す。

L910 関数 flatten_scalars

- 定義: `flatten_scalars(prefix, payload, rows)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `append, flatten_scalars, isinstance, items, str`
 - 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
1. 条件 `isinstance(payload, dict)` を判定し、真なら内部処理を行う。
 2. 条件 `isinstance(payload, (int, float, bool, str))` を判定し、真なら内部処理を行う。

L920 関数 build_current_asset_manifests

- 定義: `build_current_asset_manifests(profile_yaml, fit_summary, output_dir)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `DataFrame, append, dataframe_to_markdown, flatten_scalars, get, isinstance, isoformat, items, join, now, read_yaml, repo_relative`
 - 戻り値の要点: `{'files_csv': files_csv, 'scalars_csv': scalars_csv, 'markdown': md_path}`
1. `profile_cfg` に `read_yaml(profile_yaml)` の結果を代入する。
 2. `paths_cfg` に `profile_cfg.get('paths', {})` if `isinstance(profile_cfg, dict)` else `{}` の結果を代入する。
 3. `file_rows` に `[]` の結果を代入する。
 4. `sorted(paths_cfg.items())` を順に走査し、各要素を `(role, rel_path)` に入れて処理する。
 5. `files_df` に `pd.DataFrame(file_rows)` の結果を代入する。
 6. `files_csv` に `output_dir / 'current_active_files.csv'` の結果を代入する。
 7. `files_df.to_csv(...)` を実行する。
 8. `scalar_rows` に `[]` を代入する。
 9. `('model', 'mpc', 'runtime', 'simulation', 'live', 'measurement')` を順に走査し、各要素を `section_name` に入れて処理する。
 10. `local_fit_rows` に `[]` を代入する。

L986 関数 `render_report`

- 定義: `render_report(output_dir, source_profile_yaml, eval_profile_yaml, fit_summary, baseline_result, tuned_result, trials_df, best_weights, manifest_paths, removed_reference, specs, search_cfg, tensorboard_dir)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `Path`, `Series`, `UpperCostConfig`, `abs`, `active_upper_cost_terms`, `append`, `arange`, `as_posix`, `bar`, `close`, `compile_tex`
 - 戻り値の要点: `(tex_path, md_path) / latex_escape(os.path.relpath(path, report_dir)).replace('%', '\\%')`
1. `score_label` に 'robust score' if `str(search_cfg.get('scenario_mode', 'nominal')) == 'robust'` else 'aggregate score' の結果を代入する。
 2. `report_dir` に `output_dir / 'report'` の結果を代入する。
 3. `ensure_dir(...)` を実行する。
 4. `source_profile_label` に `repo_relative(source_profile_yaml)` の結果を代入する。
 5. `eval_profile_label` に `repo_relative(eval_profile_yaml)` の結果を代入する。
 6. `removed_reference_label` に `repo_relative(removed_reference)` or `'(none)'` の結果を代入する。
 7. `files_csv_label` に `repo_relative(manifest_paths['files_csv'])` の結果を代入する。
 8. `scalars_csv_label` に `repo_relative(manifest_paths['scalars_csv'])` の結果を代入する。
 9. `markdown_label` に `repo_relative(manifest_paths['markdown'])` の結果を代入する。
 10. `best_weights_csv_label` に `repo_relative(output_dir / 'best_upper_cost.csv')` の結果を代入する。

L1504 関数 `main`

- 定義: `main()`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `DataFrame`, `Path`, `RuntimeError`, `ScenarioRiskConfig`, `SummaryWriter`, `UpperCostConfig`, `add_argument`, `append`, `array`, `bool`, `build_current_asset_manifests`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `ap` に `argparse.ArgumentParser()` の結果を代入する。
 2. `ap.add_argument(...)` を実行する。
 3. `ap.add_argument(...)` を実行する。
 4. `ap.add_argument(...)` を実行する。
 5. `ap.add_argument(...)` を実行する。
 6. `ap.add_argument(...)` を実行する。
 7. `ap.add_argument(...)` を実行する。
 8. `ap.add_argument(...)` を実行する。
 9. `ap.add_argument(...)` を実行する。
 10. `ap.add_argument(...)` を実行する。

CLI 引数

行	引数
1506	--profile_yaml
1507	--output_profile_yaml
1508	--generations
1509	--population
1510	--elite_count
1511	--validation_top_k
1512	--elite_medium_top_k
1513	--seed
1514	--include_progress_terms
1515	--scenario-mode
1516	--coarse-upper-max-iter
1517	--elite-medium-upper-max-iter
1518	--validation-upper-max-iter
1519	--risk-cvar-alpha
1520	--risk-cvar-weight
1521	--risk-variance-weight
1522	--max-scenario-failure-probability
1523	--infeasible-score
1524	--fidelity-mode

処理の流れ

1. CLI 引数を解釈し、main() から処理を起動する。

このファイルの位置づけ

この資料群では、scripts/tune_upper_planner_weights.py を **planning research** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の profile / launch / telemetry と後段の planner / logger / report へ繋がる。